

Flink SQL reports

Seven deterministic reports (plus an optional eighth, AI-generated one — see [Optional: AI root-cause analysis](#)) that read two streams of isotope metadata — the **produce side** (the `x-isotope-*` headers stamped on every event topic record by `IsotopeProducerInterceptor`) and the **consume side** (the value-less marker records on `isotope_consume_edge_markers` emitted by `IsotopeContext.recordConsume`) — and surface what's flowing where, how fast, and how reliably. Each report runs as a long-lived `INSERT INTO <report>_1m SELECT ...` streaming job. The aggregation logic is identical across runtimes; only the source/sink DDL and function-registration glue differ.

The headline addition is the `bipartite_topology` report: it unions the produce and consume views to render the pipeline as a literal **bipartite graph** from graph theory — services in one vertex set, topics in the other, edges crossing between them in both directions. See [root README §1.0 "Background — why 'bipartite'?"](#) for the full motivation.

Runtime	Reports	Sink format	Where the SQL lives
Confluent Platform Flink (cp-flink 2.1.2 session cluster on Minikube)	7 (latency, topology, bipartite-topology, hop-distribution, coverage, stuck-trace, latency-percentiles)	<code>avro-confluent</code> (SR-framed Avro)	<code>scripts/flink/sql/cp/</code> — applied by <code>scripts/deploy-cp-flink-reports.sh</code>
Confluent Cloud for Apache Flink (CCAF)	7 (same set as CP)	<code>proto-registry</code> (SR-framed Protobuf)	Inlined as <code>confluent_flink_statement</code> resources in <code>terraform/setup-confluent-flink.tf</code> — applied by <code>scripts/deploy-cc-flink-reports.sh</code>

Control Center deserializes both sink formats natively.

What each report computes

Every report aggregates over a `TUMBLE(event_time, INTERVAL '1' MINUTE)` window. The "Source view" column names the typed view (or views) the report reads from; the views in turn filter the single `isotope_raw` source table by the presence/absence of the `x-isotope-consumer-service` header.

Every report also carries a `pipeline` column (decoded from the `x-isotope-pipeline` header) as a leading grouping dimension, so rows for an `orders` pipeline and a `location` pipeline never aggregate together. The "What it computes" column below lists the *additional* keys each report groups by, on top of `pipeline`.

Report	Source view	What it computes	Runtimes
<code>latency</code>	<code>isotope</code>	avg / min / max end-to-end latency by <code>origin_service</code> ×	CP, CCAF

Report	Source view	What it computes	Runtimes
		<code>current_topic</code>	
<code>topology</code>	<code>isotope</code>	produce-edge counts per <code>(producer_service, topic)</code> per minute	CP, CCAF
<code>bipartite_topology</code>	<code>isotope</code> and <code>consume_events</code>	full bipartite graph: produce edges plus consume edges per minute	CP, CCAF
<code>hop_distribution</code>	<code>isotope</code>	record counts bucketed by <code>hop_count</code> per topic per minute	CP, CCAF
<code>coverage</code>	<code>isotope</code>	distinct traces per topic per minute	CP, CCAF
<code>stuck_trace</code>	<code>isotope</code>	alerts (via <code>STUCK_TRACE_PTF</code>) for traces idle ≥ 60 s of event time	CP, CCAF
<code>latency_percentiles</code>	<code>isotope</code>	p50 / p95 / p99 (via <code>LATENCY_PERCENTILES</code> PTF, T-Digest)	CP, CCAF

Optional: AI root-cause analysis (CCAF-only, off by default)

Beyond the seven deterministic reports, an **eighth, AI-generated report** turns each stuck-trace *alert* into a natural-language root-cause hypothesis plus a one-line remediation. It's a CCAF-only feature, wired by [terraform/setup-ccaf-ai.tf](#) and **gated on `var.enable_trace_rca` (default `false`)**, so a normal deploy is unaffected.

When enabled, it adds three `confluent_flink_statement` resources:

- `CREATE MODEL trace_rca` — registers a remote text-generation model.
- A `proto-registry` sink table `isotope_report_trace_rca_1m`.
- An `INSERT ... SELECT ... LATERAL TABLE(ML_PREDICT('trace_rca', ...))` that calls the model **once per alert** (reading the low-volume 1-minute stuck-trace report topic, never the source stream) and writes the hypothesis to its own topic — it never overwrites the deterministic reports.

The default provider is OpenAI (`gpt-4o`). Claude is supported two ways: directly via **Anthropic** (`rca_model_provider = "anthropic"`, endpoint `https://api.anthropic.com/v1/messages`, a bare `rca_model_api_key`, plus the required `rca_model_max_tokens`), or via **AWS Bedrock** (`rca_model_provider = "bedrock"` with AWS credentials). Other supported providers: `vertexai`, `azureopenai`, `googleai`, `sagemaker`, `azureml`.

The standalone SQL walkthrough — a `CREATE MODEL + ML_PREDICT` PoC with provider notes and alternative options — is in [sql/ccaf-ai/trace_rca.fql](#). See also [root README §4.5](#).

Format-by-runtime, not by domain

The sink **format** differs by runtime for one platform-level reason:

- **cp-flink ships SR-Avro but not SR-Protobuf.** Apache Flink open-source publishes `flink-sql-avro-confluent-registry`; there is no SR-integrated Protobuf equivalent. We could hand-write one (~150 lines wrapping `flink-protobuf` with magic-byte framing
 - SR client) but Avro already gives us Control Center decoding with zero custom code. CCAF has SR-Protobuf, so its sinks use it.

The report **set** is identical on both runtimes — but that took a deliberate choice for percentiles: **CCAF rejects all user-defined aggregate functions** with `aggregate functions are not supported`, regardless of accumulator shape. So `LATENCY_PERCENTILES` is implemented as a `ProcessTableFunction` (T-Digest sketch + per-window state and timers), not an aggregate function — a PTF registers and runs on both runtimes, same as `STUCK_TRACE_PTF`.

The demo event topics (`orders.placed`, `orders.enriched`, `orders.fulfilled`) ride Protobuf+SR via the Java app's `DemoEvent` schema on both runtimes — those are written by the Kafka producer client, not by Flink, so the SR-Protobuf gap doesn't apply.

Layout

<code>scripts/flink/sql/cp/</code>	CP Flink – session-cluster SQL
<code>00_source_table.fql</code>	CREATE TABLE per topic ('connector'
<code>= 'kafka') + isotope_raw UNION view</code>	
<code>01_register_functions.fql</code>	CREATE FUNCTION ... USING JAR
<code>'file:///opt/flink/lib/isotope-flink-udf.jar'</code>	
<code>05_isotope_view.fql</code>	Typed view; decodes x-isotope-*
<code>header scalars (produces only)</code>	
<code>06_consume_events_view.fql</code>	Typed view of
<code>isotope_consume_edge_markers markers (consume edges)</code>	
<code>05_report_sinks.fql</code>	CREATE TABLE for each
<code>isotope_report_*_1m Kafka sink (avro-confluent)</code>	
<code>10_latency_report.fql</code>	INSERT INTO: avg/min/max latency by
<code>origin × topic</code>	
<code>20_topology_report.fql</code>	INSERT INTO: produce-edge counts
<code>per minute</code>	
<code>25_bipartite_topology_report.fql</code>	INSERT INTO: produce plus consume
<code>edges (bipartite graph) per minute</code>	
<code>30_hop_distribution.fql</code>	INSERT INTO: hop-count buckets per
<code>topic per minute</code>	
<code>40_coverage_report.fql</code>	INSERT INTO: distinct traces per
<code>topic per minute</code>	
<code>60_stuck_trace_report.fql</code>	INSERT INTO: stuck-trace alerts via
<code>STUCK_TRACE_PTF</code>	
<code>70_latency_percentiles_report.fql</code>	INSERT INTO: p50/p95/p99 via
<code>LATENCY_PERCENTILES PTF (T-Digest)</code>	
<code>99_teardown.fql</code>	DROP TABLE/VIEW/FUNCTION (companion
<code>to flink-reports-down)</code>	
 <code>scripts/flink/sql/ccaf-ai/</code>	 Optional AI report – CCAF only (see
<code>section below)</code>	
<code>trace_rca.fql</code>	CREATE MODEL + ML_PREDICT PoC: LLM
<code>root-cause hypothesis per stuck-trace alert</code>	

CCAF runs the same seven reports, but the SQL lives inline as `confluent_flink_statement` resources in `terraform/setup-confluent-flink.tf` — 25 statements: ALTER (×4) + raw view + typed produce view + typed consume view + 7 sinks + `STUCK_TRACE_PTF` and `LATENCY_PERCENTILES` drop+register (×2 each = 4) + 7 INSERTs. The JAR is uploaded as a `confluent_flink_artifact` and referenced via `USING JAR 'confluent-artifact://<id>'`.

Wire-format detail (CP only)

Window columns ride as `BIGINT` epoch millis on the wire, not `TIMESTAMP_LTZ`. Flink 2.1.2's `avro-confluent` schema-derivation path raises `UnsupportedOperationException: Unsupported to derive Schema for type: TIMESTAMP_LTZ(3)` for that type. We cast in the INSERT `(UNIX_TIMESTAMP(CAST(window_start AS STRING)) * 1000)` so the on-wire schema is plain Avro `long`. Consumers rehydrate via `TO_TIMESTAMP_LTZ(window_start, 3)`. The CCAF Protobuf sinks don't hit this — `proto-registry` handles `TIMESTAMP_LTZ` directly.

PTF JAR

The single shadow JAR `ptf/build/libs/isotope-flink-udf.jar` (produced by `./gradlew :ptf:shadowJar`) carries both JAR-backed functions — both `ProcessTableFunctions` — under the `ai.signalroom.kafka.isotope.flink` package:

- `LatencyPercentilesPTF` (registered as `LATENCY_PERCENTILES`) — T-Digest p50/p95/p99 via per-window state + event-time timers.
- `StuckTracePTF` — per-trace state + event-time timer that emits alerts for traces idle ≥60s.

Both register on both runtimes; only the `CREATE FUNCTION ... USING JAR ...` clause differs (`file://` path on CP, `confluent-artifact://` reference on CCAF).

Operations

CP Flink (Minikube):

```
make flink-up          # cert-manager → CFK Flink Operator → CMF → session
                        cluster
make flink-reports-up  # build PTF JAR, copy to JM, create sink topics,
                        submit 7 INSERT INTO jobs
make flink-sql         # interactive SQL Client (auto-loads sink DDL so
                        SELECT * works)
make flink-reports-down # cancel jobs, drop tables, delete sink topics + SR
                        subjects
make flink-down        # tear down cluster + operator + cert-manager
```

CCAF (Confluent Cloud, Terraform-driven):

```
make cc-flink-reports-up  CONFLUENT_API_KEY=... CONFLUENT_API_SECRET=...
                        # terraform apply: env + cluster + topics +
compute pool + artifact + 25 statements
                        # also regenerates terraform/terraform.png via
```

```
`terraform graph | dot`
source scripts/cc-cli-env.sh          # exports BOOTSTRAP / SR_URL /
KAFKA_KEY / KAFKA_SECRET / SR_KEY / SR_SECRET / JAAS
scripts/cc-app-run.sh send orders.placed order-intake-service 'hello' #
drives traffic with the SASL config
make cc-flink-reports-down CONFLUENT_API_KEY=... CONFLUENT_API_SECRET=...
                                     # terraform destroy: deletes the environment and
everything in it
```

See the [root README §4.5](#) for the full CCAF walkthrough, including the multi-window sustained-traffic pattern required to see tumbling-window aggregates emit.